

PROJECT 3 - IMPLEMENT CLIENT SCRIPT & UI POLICY (INCIDENT)

ABSTRACT

Incident records need consistent and accurate data so that they can be triaged, routed and resolved quickly. When data entry depends only on the awareness of the user, incomplete or incorrect records are submitted, which affects reporting accuracy, SLA compliance and service quality. This project, "**Implement Client Script & UI Policy (Incident)**", shows how client-side controls in ServiceNow can enforce business rules directly on the Incident form before a record is saved.

A UI Policy named **High Impact Control** was created on the Incident table with the condition **Impact is 1 - High** and "Reverse if false" enabled. Its UI Policy Actions make the **Assignment group** field mandatory and the **Urgency** field read-only while the condition is true. Three client scripts were then added: an **onChange** script that automatically sets Urgency to High when Impact is changed to High, an **onSubmit** script that blocks saving a High impact incident when **Assigned to** is empty, and an **onCellEdit** script that prevents the **State** field from being changed by editing directly in a list.

The configuration was tested on a new incident (INC0010012) and on existing incidents. Saving without the Assigned to value displayed the error "Assigned To is mandatory for High impact incidents"; saving with a user selected succeeded; changing Impact back to Medium reversed the policy; and a State edit from the list was blocked with an alert, while the same change made on the form was saved. The project shows how UI Policies and Client Scripts work together to improve data quality and user guidance without server-side changes.

Keywords: ServiceNow, Incident Management, UI Policy, UI Policy Action, Client Script, onChange, onSubmit, onCellEdit, Form Validation.

1. INTRODUCTION

1.1 Overview

An incident is an unplanned interruption or reduction in the quality of an IT service. The Incident form is used by service desk agents and end users to record what happened, who is affected, how serious it is and who should work on it. If fields such as impact, urgency and assignment are missing or inconsistent, the incident may be routed to the wrong team and may breach its SLA.

ServiceNow provides two client-side tools to control the behaviour of a form in the browser: **UI Policies**, which change the visibility, mandatory and read-only state of fields when conditions are met, and **Client Scripts**, which run JavaScript when a form loads, a field changes, a form is submitted or a list cell is edited. This project uses both tools on the Incident table.

1.2 About the Naan Mudhalvan Programme

Naan Mudhalvan is a flagship skilling initiative of the Government of Tamil Nadu, implemented by the Tamil Nadu Skill Development Corporation (TNSkills). It provides students of arts, science and engineering colleges with industry-aligned training and certification in emerging technologies so that they become employable. This project has been carried out as part of the ServiceNow course offered under the programme.

1.3 Problem Statement

Incident records often require consistent and accurate data entry to ensure effective triage, routing and resolution. Relying solely on user awareness and manual checks can lead to incomplete, inconsistent or incorrect data being submitted, which affects reporting accuracy, SLA compliance and overall service quality. There is a need to enforce conditional field behaviour and validation directly at the user interface level.

1.4 Objectives

1. Demonstrate how client-side controls in ServiceNow enforce data integrity on Incident records.
2. Use a UI Policy and UI Policy Actions to make fields mandatory or read-only when the Impact is High.
3. Use Client Scripts to auto-populate values, validate the form on save and control editing from lists.

4. Prevent submission of an incident when the required conditions are not met.
5. Verify the behaviour, including the reverse behaviour when the condition is no longer true.

1.5 Skills Covered

- Incident Management
- UI Policy and UI Policy Actions
- Client Scripts (onChange, onSubmit, onCellEdit)
- Form validation

1.6 Scope of the Project

The project covers the Incident table in the Global application scope and one business rule: incidents with High impact. Server-side controls such as Business Rules and Data Policies, and mobile or Service Portal specific behaviour, are outside the scope of this work.

2. SYSTEM STUDY

2.1 Existing System

- Users are expected to know which fields must be filled for a high impact incident.
- Manual checks by supervisors are needed to find missing assignment information.
- Urgency can be set inconsistently with the impact, so priority and reports are unreliable.
- State values can be changed quickly from a list without the context of the full form.

2.2 Proposed System

The proposed system enforces the rules directly in the browser:

- A UI Policy makes **Assignment group** mandatory and **Urgency** read-only when Impact is High, and reverses these actions when it is not.
- An onChange client script sets Urgency to High automatically when Impact becomes High.
- An onSubmit client script stops a High impact incident from being saved without an Assigned to value and shows an error on the field.
- An onCellEdit client script stops the State field from being changed in a list and asks the user to open the incident.

2.3 Feasibility Study

- **Technical feasibility:** UI Policies and Client Scripts are standard ServiceNow features and need no extra installation.
- **Economic feasibility:** Only configuration and small scripts are required, so the cost is very low.
- **Operational feasibility:** The rules are visible to the user on the form, so little training is needed.

3. SYSTEM REQUIREMENTS

3.1 Hardware Requirements

Table 3.1: Hardware requirements

Component	Specification
Processor	Intel Core i3 or above (or equivalent)
RAM	4 GB minimum (8 GB recommended)
Storage	256 GB or above
Network	Stable internet connection

3.2 Software Requirements

Table 3.2: Software requirements

Component	Specification
Platform	ServiceNow Personal Developer Instance (PDI) / training instance
Access level	Administrator or configuration access
Web browser	Google Chrome / Microsoft Edge (latest version)
Scripting language	JavaScript (client-side, using the g_form API)
Application scope	Global

3.3 ServiceNow Components Used

Table 3.3: ServiceNow components and their role

Component	Navigation / table	Role in the project
UI Policy	System UI > UI Policies	Conditional mandatory and read-only behaviour
UI Policy Action	Related list of the UI Policy	Field-level actions for Assignment group and Urgency
Client Script	System Definition > Client Scripts	onChange, onSubmit and onCellEdit scripts
Incident	incident	Table on which all rules apply
g_form	Client-side API	Used to read and set field values and show messages

4. SYSTEM DESIGN

4.1 Business Rules to Implement

Table 4.1: Business rules and the mechanism used

No.	Business rule	Mechanism
1	When Impact is High, Assignment group must be filled	UI Policy action (Mandatory = True)
2	When Impact is High, Urgency cannot be edited by the user	UI Policy action (Read only = True)
3	When Impact is changed to High, Urgency becomes High	onChange client script on Impact
4	A High impact incident cannot be saved without Assigned to	onSubmit client script
5	State cannot be changed by editing in a list	onCellEdit client script on State
6	When Impact is not High, the UI Policy effects are removed	Reverse if false = True

4.2 UI Policy Design

Table 4.2: Design of the UI Policy "High Impact Control"

Setting	Value
Short description (name)	High Impact Control
Table	Incident [incident]
Active	True
Condition	Impact is 1 - High
Reverse if false	True
On load	True
Global	True
UI Policy Action 1	Assignment group: Mandatory = True, Read only = False, Visible = Leave alone
UI Policy Action 2	Urgency: Mandatory = Leave alone, Read only = True, Visible = Leave alone

4.3 Client Script Design

Table 4.3: Design of the client scripts

Script name	Type	Field	Purpose
Auto set urgency for high impact	onChange	Impact	Sets Urgency to 1 (High) and shows a message when Impact is High

Script name	Type	Field	Purpose
Prevent save if Assigned To missing	onSubmit	-	Blocks save when Impact is High and Assigned to is empty
Prevent state change via list edit	onCellEdit	State	Shows an alert and cancels the edit made in a list

4.4 Expected Behaviour

Table 4.4: Expected behaviour of the Incident form

Situation	Form behaviour	Source
Impact set to High	Urgency set to High; Urgency read-only; Assignment group mandatory	onChange script and UI Policy
Save with Impact High and empty Assigned to	Save blocked; error shown on Assigned to	onSubmit script
Save with Impact High and Assigned to filled	Record saved	onSubmit script
Impact changed from High to Medium	Urgency editable; Assigned to no longer required	Reverse if false
State edited in a list	Alert shown; value unchanged	onCellEdit script
State edited on the form	Change saved	No restriction on the form

5. IMPLEMENTATION

The implementation was carried out in the ServiceNow instance with administrative access, in six tasks: creating the UI Policy and its actions, creating three client scripts, and testing the configuration.

5.1 Task 1 - Create the UI Policy on Incident

Activity 1: Create the UI Policy

1. Log in to the ServiceNow instance with administrative or configuration access.
2. Navigate to **System UI > UI Policies**.
3. Click **New** to create a new UI Policy record.

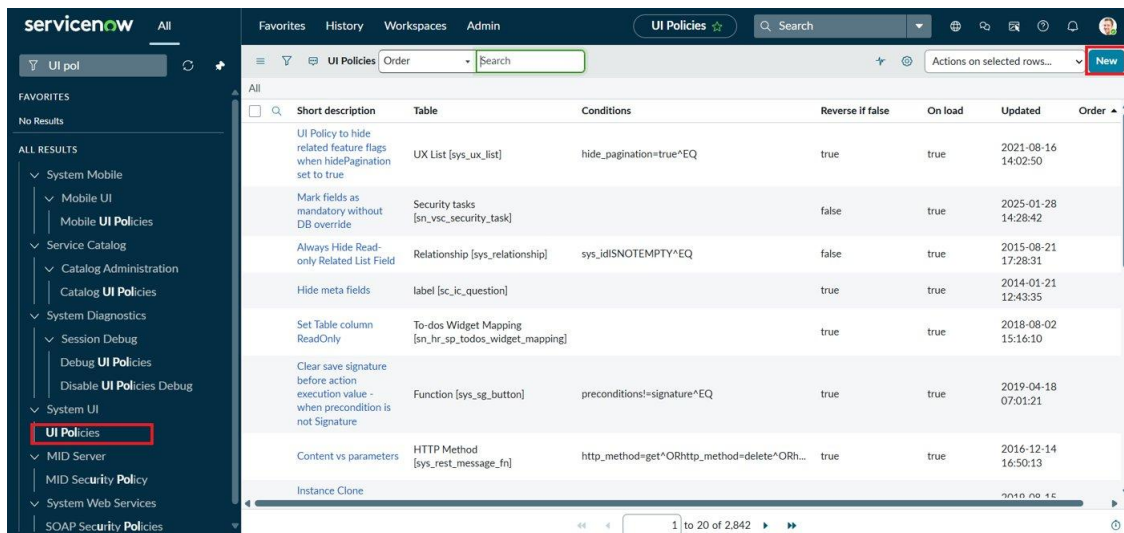


Figure 1: System UI > UI Policies list with the New button

4. Enter the Short description (name) **High Impact Control**. The name shows that the policy adds controls when an incident has a high impact.
5. Select the Table **Incident [incident]**, so that the policy applies to Incident records.
6. Keep **Active** selected so that the policy is enforced on the Incident form.
7. In the Conditions section, choose Field **Impact**, Operator **is**, Value **1 - High**. The policy is triggered only when the impact is High.
8. Select **Reverse if false**, so that when the condition is not met, the changes applied by the policy are reverted.
9. Review the configuration and click **Submit**.

The screenshot shows the ServiceNow UI Policy configuration interface. The left sidebar lists various system components, including 'UI Policies'. The main area displays the configuration for a policy named 'High Impact Control'. The 'When to Apply' tab is active, showing a condition where 'Impact' is '1 - High'. The 'Reverse if false' checkbox is checked. The 'Table' is set to 'Incident [incident]'. The 'Short description' is 'High Impact Control' and the 'Order' is 100. The 'Application' is set to 'Global' and 'Active' is checked. The 'On load' checkbox is also checked. The 'Reverse if false' checkbox is checked. The 'Inherit' checkbox is unchecked.

Figure 2: UI Policy "High Impact Control" - table, condition and Reverse if false

10. Open the saved policy and, in the **UI Policy Actions** related list, create an action for the **Assignment group** field.
11. Set **Mandatory** to **True** and **Read only** to **False**, and save.

The screenshot shows the ServiceNow UI Policy Action configuration interface. The left sidebar lists various system components, including 'UI Policies'. The main area displays the configuration for a policy action named 'assignment_group'. The 'Field name' is set to 'Assignment group'. The 'Mandatory' checkbox is checked. The 'Read only' checkbox is checked. The 'Visible' checkbox is unchecked. The 'Clear the field value' checkbox is unchecked. The 'Update' and 'Delete' buttons are visible at the bottom.

Figure 3: UI Policy Action for Assignment group (Mandatory = True)

5.2 Task 2 - Create the UI Policy Action for Urgency

Activity 1: Configure the Urgency field

1. Open the **High Impact Control** UI Policy and scroll down to the **UI Policy Actions** related list.
2. Click **New** to create a UI Policy Action.
3. Select the Field name **Urgency**.
4. Set **Read only** to **True**. Users can see the field but cannot change its value while Impact is High.

5. Leave **Visible** as **Leave alone**, so the field stays visible on the form.
6. Click **Submit** to save the action. It is now linked to the High Impact Control policy.

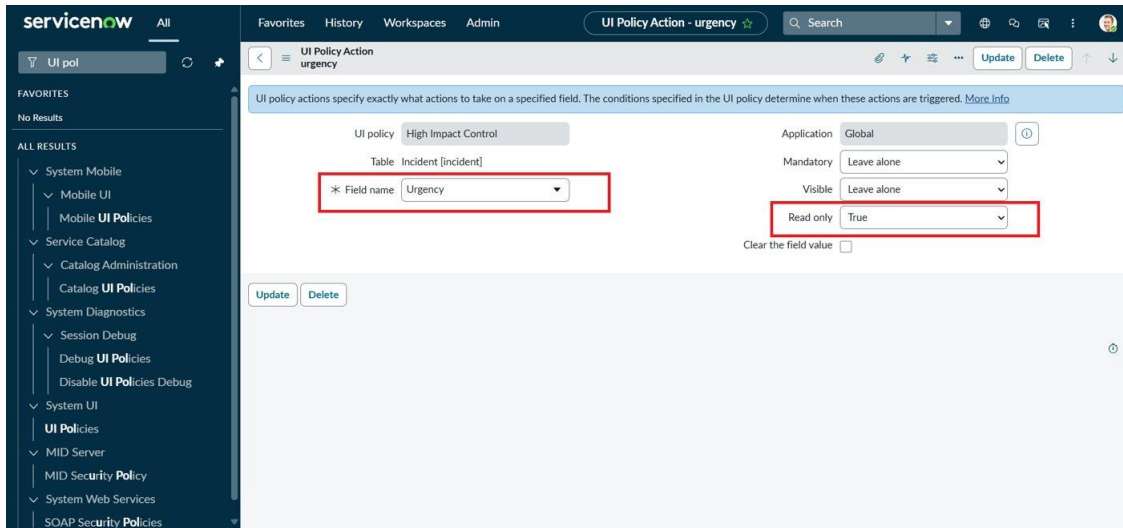


Figure 4: UI Policy Action for Urgency (Read only = True)

Optional verification: open an incident, set Impact to High and check that Urgency becomes read-only; change Impact to a different value and confirm that the field reverts.

5.3 Task 3 - Create the onChange Client Script

Activity 1: Create the client script

1. Navigate to **System Definition > Client Scripts** (System UI in the navigator) and click **New**.

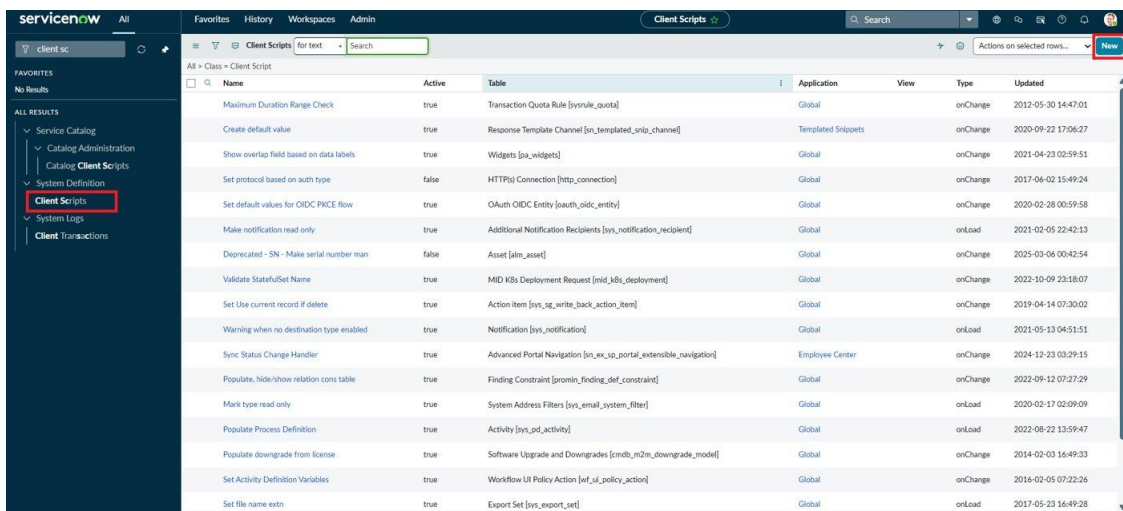


Figure 5: Client Scripts list with the New button

2. Enter the Name **Auto set urgency for high impact**.
3. Select the Table **Incident [incident]**, so the script runs only on Incident records.

4. Select the Type **onChange** and the Field name **Impact**, so the script runs when Impact is updated.
5. Keep **Active** selected.
6. Paste the following script into the script editor and click **Submit**.

```
function onChange(control, oldValue, newValue, isLoading) {
    if (isLoading || newValue == '') {
        return;
    }
    if (newValue == '1') {
        g_form.setValue('urgency', '1');
        g_form.addInfoMessage('Urgency set to High for High impact incident.');
```

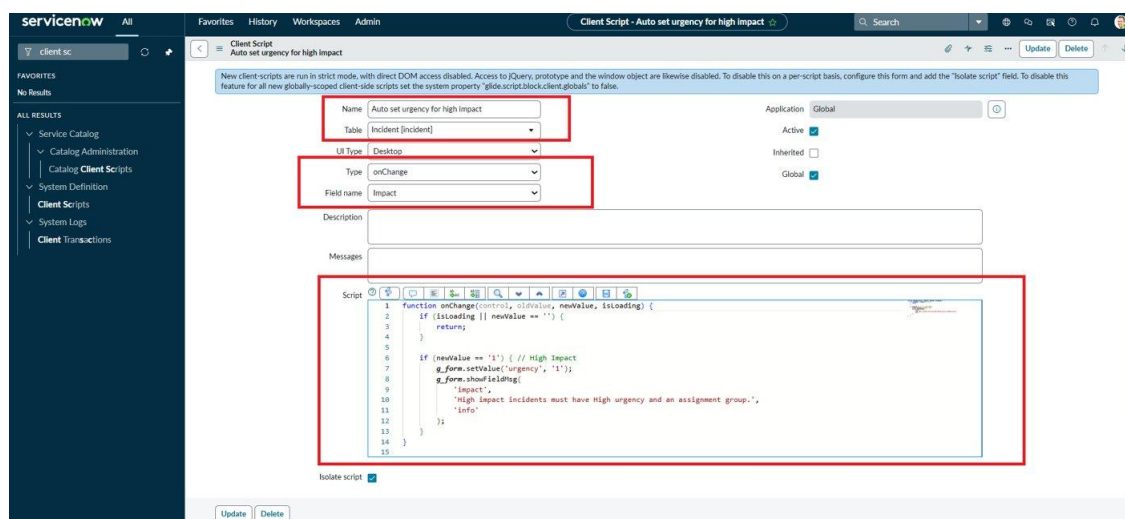


Figure 6: onChange client script "Auto set urgency for high impact"

Verification: open an incident and change the Impact field to High. The Urgency field is updated automatically.

5.4 Task 4 - Create the onSubmit Client Script (Save Validation)

Activity 1: Create the client script

1. Navigate to **System Definition > Client Scripts** and click **New**.
2. Enter the Name **Prevent save if Assigned To missing**.
3. Select the Table **Incident [incident]** and the Type **onSubmit**, which runs when the user tries to save the record.
4. Keep **Active** selected so that the script is enforced immediately.
5. Paste the following script and click **Submit**.

```
function onSubmit() {
```

```

if (g_form.getValue('impact') == '1' &&
    g_form.getValue('assigned_to') == '') {
    g_form.showErrorMessage(
        'assigned_to',
        'Assigned To is mandatory for High impact incidents.'
    );
    return false;
}
return true;
}

```

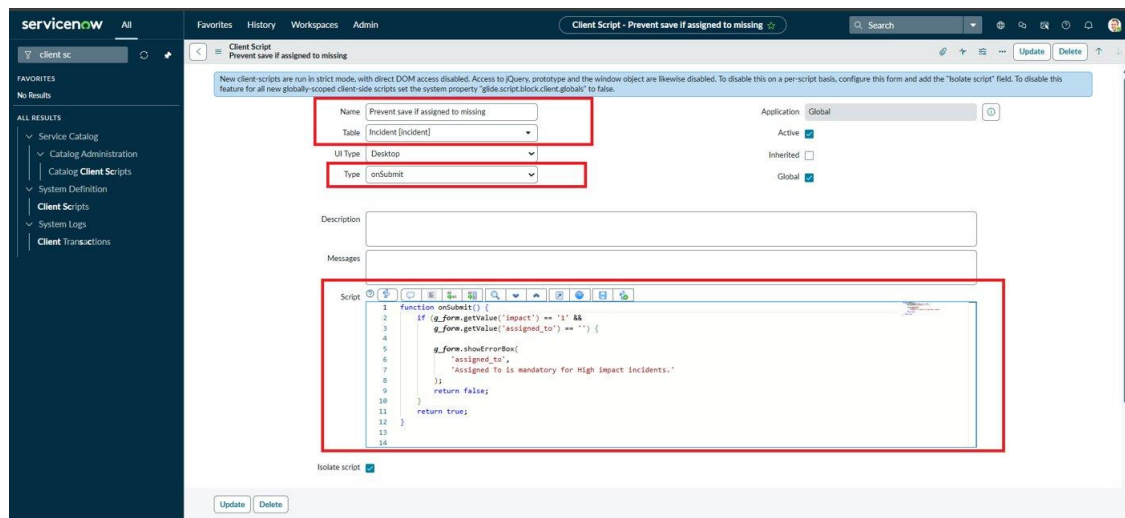


Figure 7: onSubmit client script "Prevent save if Assigned To missing"

Verification: try saving a High impact incident without a value in Assigned to. The save is prevented and an error is shown.

5.5 Task 5 - Create the onCellEdit Client Script

Activity 1: Create the client script

1. Navigate to **System Definition > Client Scripts** and click **New**.
2. Enter the Name **Prevent state change via list edit**.
3. Select the Table **Incident [incident]**, the Type **onCellEdit** and the Field name **State**.
4. Keep **Active** selected.
5. In the Script field, paste the following code and click **Submit**.

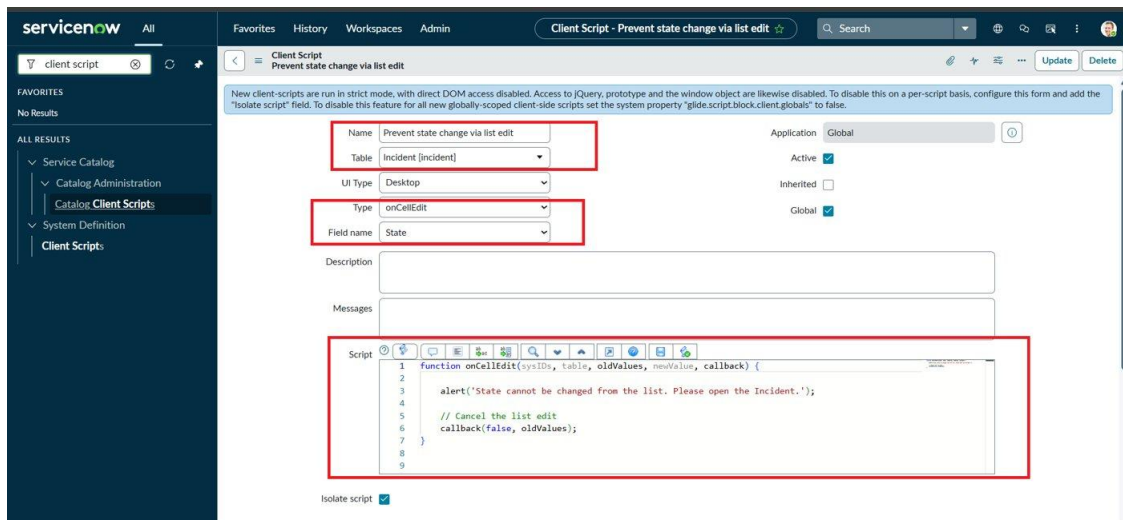
```

function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
    alert('State cannot be updated using list editing. Please open the Incident.');
```

```

    callback(false);
}

```



Verification: open an Incident list and try to change the State directly in the list. The script blocks the change.

5.6 Task 6 - Testing the Configuration

Activity 1: Test mandatory enforcement

1. Go to **Incident > Create New** to open a new Incident form.

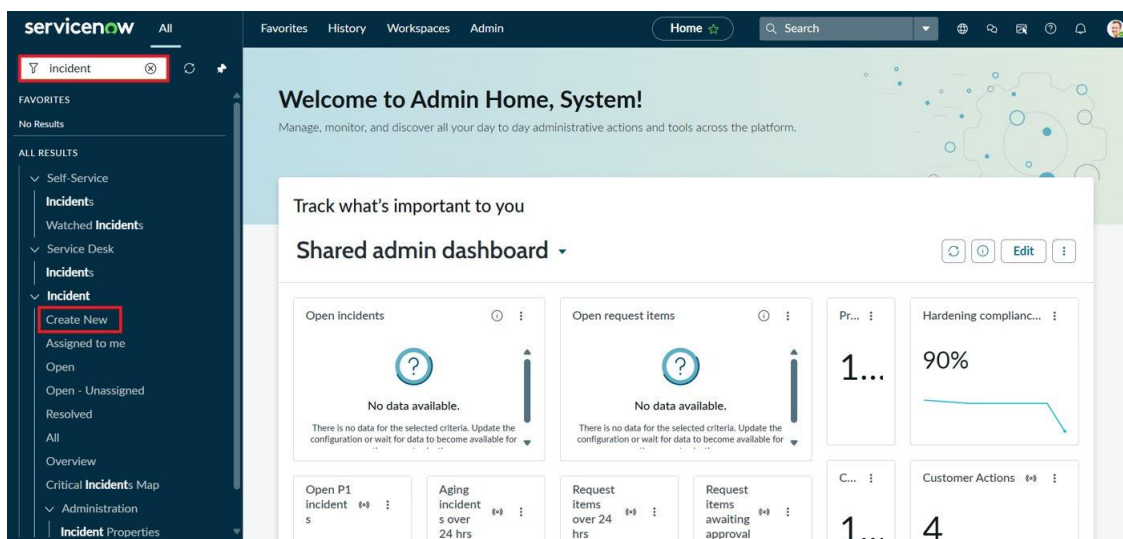


Figure 9: Opening Incident > Create New

2. Select **1 - High** for the Impact field. This triggers the UI Policy and the onChange client script.
3. Enter a Short description and leave **Assigned to** empty.
4. Click **Submit** to try to save the record.

- The incident is not saved, and the error message "Assigned To is mandatory for High impact incidents." appears under the Assigned to field. Urgency is set to High and is read-only, and Assignment group is mandatory.

The screenshot shows the 'Incident - Create INC0010012' form in ServiceNow. The form is partially filled out with the following values: Number: INC0010012, Caller: Adela Cervantsz, Category: Inquiry / Help, Subcategory: -- None --, Service: -- None --, Service offering: -- None --, Configuration item: -- None --, Channel: -- None --, State: New, Impact: 1 - High, Urgency: 1 - High, Priority: 1 - Critical, Assignment group: Software, and Assigned to: (empty). The 'Assigned to' field is highlighted with a red box, and a red error message is displayed below it: "Assigned To is mandatory for High Impact incidents." The 'Submit' button is also highlighted with a red box.

Figure 10: Error message when saving a High impact incident without Assigned to

Activity 2: Test successful save

- On the same form, select a user in the **Assigned to** field.
- Click **Submit**.
- The incident is saved without errors, and the Urgency auto-setting and read-only behaviour remain in effect.

The screenshot shows the 'Incident - Create INC0010012' form in ServiceNow. The form is partially filled out with the following values: Number: INC0010012, Caller: Adela Cervantsz, Category: Inquiry / Help, Subcategory: -- None --, Service: -- None --, Service offering: -- None --, Configuration item: -- None --, Channel: -- None --, State: New, Impact: 1 - High, Urgency: 1 - High, Priority: 1 - Critical, Assignment group: Software, and Assigned to: Beth Anglin. The 'Assigned to' field is highlighted with a red box, and the 'Submit' button is also highlighted with a red box. The error message from the previous figure is no longer present.

Figure 11: Successful save after selecting a user in Assigned to

Activity 3: Reverse condition test

- Go to **Incident > Open** and select an incident where Impact is High.
- Change Impact from **High** to **Medium**.

- Assigned to is no longer required, and Urgency is editable, which shows that the read-only action of the policy is reversed.
- Click **Update**. The record saves without errors.

Figure 12: Reverse condition test - Impact changed from High to Medium

Activity 4: Test list edit blocking

- Go to **Incident > All** (or Open) to view the incidents in a list.

Number	Opened	Short description	Caller	Priority	State	Category	Assignment group	Assigned to	Updated	Updated by
INC0010012	2026-01-14 02:15:14	testing UI policy	Adela Cervantsz	2 - High	New	Inquiry / Help	Software	(empty)	2026-01-14 03:19:11	admin
INC0009009	2018-08-30 01:06:16	Unable to access the shared folder.	David Miller	4 - Low	New	Inquiry / Help	(empty)	(empty)	2018-12-12 23:30:24	admin
INC0009005	2018-08-31 21:35:21	Email server is down.	David Miller	1 - Critical	New	Software	(empty)	(empty)	2018-12-12 23:18:55	admin
INC0009001	2018-09-11 20:56:26	Unable to post content on a Wiki page	David Miller	3 - Moderate	New	Inquiry / Help	(empty)	(empty)	2018-12-12 23:32:42	admin
INC0008112	2019-07-29 11:48:43	Assessment - ATF Assessor	survey user	5 - Planning	New	Inquiry / Help	(empty)	(empty)	2019-07-29 11:49:28	admin
INC0008111	2019-07-22 14:04:57	ATF: Test1	System Administrator	5 - Planning	New	Inquiry / Help	(empty)	(empty)	2019-07-22 14:05:48	admin
INC0008001	2021-01-15 13:04:14	ATF: TEST2	survey user	5 - Planning	New	Inquiry / Help	(empty)	(empty)	2021-01-21 15:31:42	admin
INC0007002	2018-10-16 22:47:51	Need access to the common drive.	David Miller	4 - Low	New	Inquiry / Help	(empty)	(empty)	2018-12-12 23:28:49	admin
INC0007001	2018-10-16 22:47:10	Employee payroll application server is down.	David Miller	1 - Critical	New	Hardware	Openspace	(empty)	2025-12-03 18:11:31	system
INC0001990	2020-06-07 09:02:25	Unable to access the personal details section in payroll portal	Problem CoordinatorATF	5 - Planning	On Hold	Inquiry / Help	(empty)	(empty)	2020-06-07 09:04:34	admin
INC0001990	2016-08-10	Unable to access team	Problem Coordinator	3 - Moderate	Resolved	Inquiry / Help	(empty)	(empty)	2016-08-10	admin

Figure 13: Incident list opened from Incident > Open

- Double-click the **State** field of an incident to edit it directly in the list.
- An alert appears stating that the State cannot be changed from the list and that the incident should be opened. The State value is unchanged.

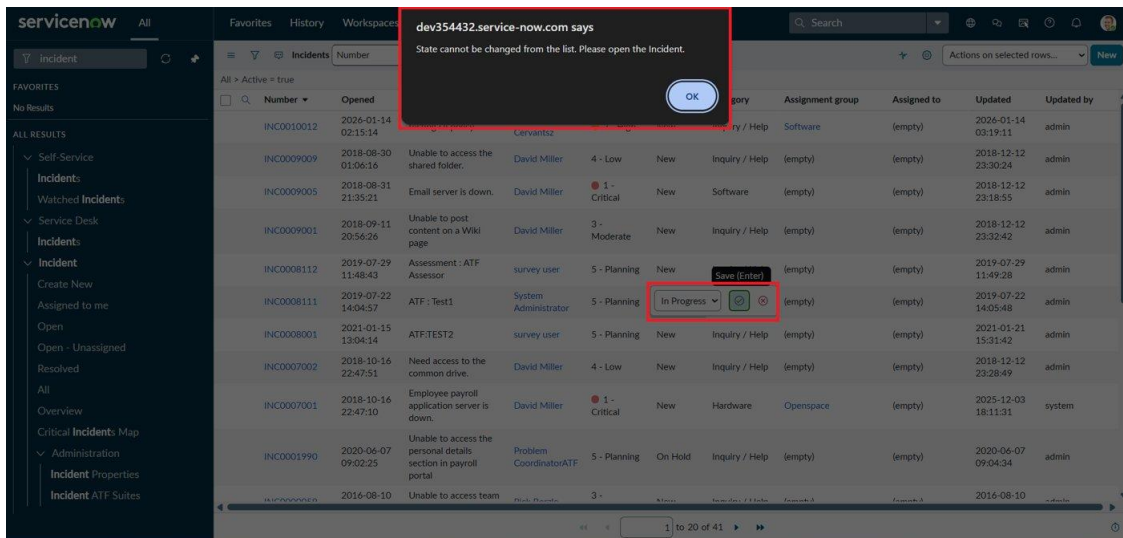


Figure 14: Alert blocking the State change made in the list

Activity 5: Test form-based update

1. Open the same incident record used in the earlier tests.
2. On the form, change the **State** to a new value (for example, In Progress) and save the record using **Update** or the Save option in the form menu.

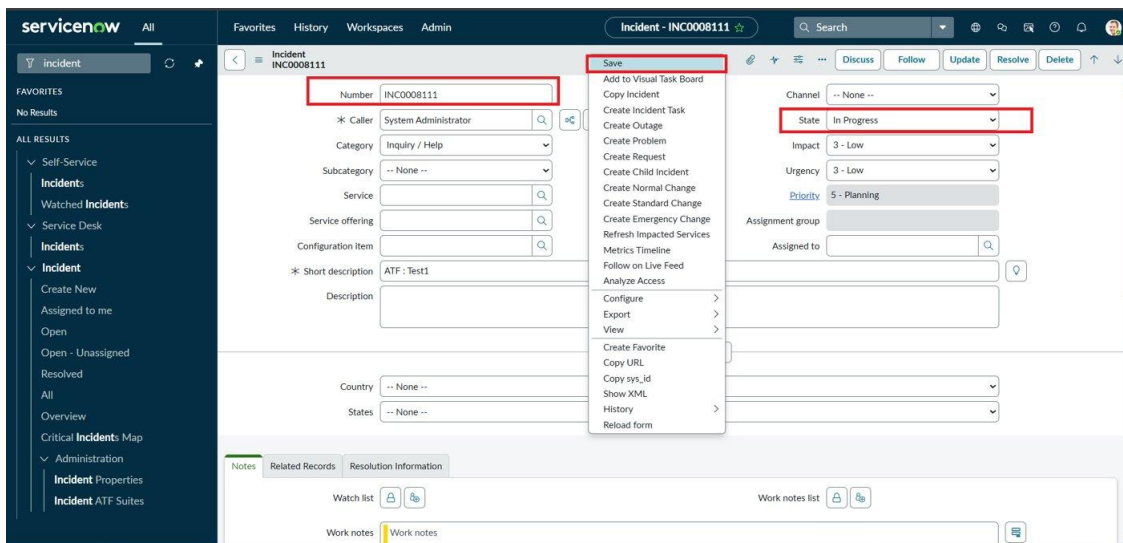


Figure 15: Changing the State on the incident form and saving

3. The State change is saved successfully, as shown in the incident list, while the other rules continue to apply.

servicenow		Incidents									
Incident		Number Search Actions on selected rows... New									
FAVORITES		All > Active = true									
No Results		Number Opened Short description Caller Priority State Category Assignment group Assigned to Updated Updated by									
ALL RESULTS		INC0010012 2024-01-14 02:15:14 testing UI policy Adela Cervantsz 2 - High New Inquiry / Help Software (empty) 2024-01-14 03:19:11 admin									
Self-Service Incidents Service Desk Incidents Incident Create New Assigned to me Open Open - Unassigned Resolved All Overview Critical Incidents Map Administration Incident Properties Incident ATF Suites		INC0009009 2018-08-30 01:06:16 Unable to access the shared folder. David Miller 4 - Low New Inquiry / Help (empty) (empty) 2018-12-12 23:30:24 admin									
		INC0009005 2018-08-31 21:35:21 Email server is down. David Miller 1 - Critical New Software (empty) (empty) 2018-12-12 23:18:55 admin									
		INC0009001 2018-09-11 20:56:26 Unable to post content on a Wiki page David Miller 3 - Moderate New Inquiry / Help (empty) (empty) 2018-12-12 23:32:42 admin									
		INC0008112 2019-07-29 11:48:43 Assessment : ATF Assessor survey user 5 - Planning New Inquiry / Help (empty) (empty) 2019-07-29 11:49:28 admin									
		INC0008111 2019-07-22 14:04:57 ATF : Test1 System Administrator 5 - Planning In Progress Inquiry / Help (empty) (empty) 2024-01-14 03:31:59 admin									
		INC0008001 2021-01-15 13:04:14 ATF:TEST2 survey user 5 - Planning New Inquiry / Help (empty) (empty) 2021-01-21 15:31:42 admin									
		INC0007002 2018-10-16 22:47:51 Need access to the common drive. David Miller 4 - Low New Inquiry / Help (empty) (empty) 2018-12-12 23:28:49 admin									
		INC0007001 2018-10-16 22:47:10 Employee payroll application server is down. David Miller 1 - Critical New Hardware Opspace (empty) 2025-12-03 18:11:31 system									
		INC0001990 2020-06-07 09:02:25 Unable to access the personal details section in payroll portal. Problem CoordinatorATF 5 - Planning On Hold Inquiry / Help (empty) (empty) 2020-06-07 09:04:34 admin									
		INC0000000 2016-08-10 Unable to access team David Miller 3 - New Hardware Opspace (empty) 2016-08-10 admin									

Figure 16: Incident list showing the State updated to In Progress

6. TESTING AND RESULTS

6.1 Test Cases

Table 6.1: Test cases and observed results

S.No	Test	Expected result	Observed result
1	Set Impact to High on a new incident	Urgency becomes High and read-only	Urgency 1 - High (read-only) - Figure 10
2	Save High impact incident with empty Assigned to	Save blocked with an error message	Error shown under Assigned to - Figure 10
3	Save High impact incident with Assigned to filled	Record saved	INC0010012 saved with Assigned to selected - Figure 11
4	Change Impact from High to Medium	Policy actions reversed	Urgency editable; Assigned to not required - Figure 12
5	Edit State in the incident list	Alert shown and value unchanged	Alert displayed - Figure 14
6	Change State on the incident form	Change saved	State saved as In Progress - Figures 15, 16

6.2 Result Analysis

All six tests produced the expected results. The UI Policy controlled the mandatory and read-only behaviour and reversed it correctly, the onChange script kept Urgency consistent with Impact, the onSubmit script prevented incomplete high impact incidents from being saved, and the onCellEdit script protected the State field from list editing while still allowing changes on the form.

7. ADVANTAGES AND LIMITATIONS

7.1 Advantages

- Users get immediate feedback on the form, so errors are corrected before saving.
- Improves the quality and consistency of incident data, and the accuracy of reports and SLAs.
- Lightweight and easy to implement using configuration and short scripts.
- The UI Policy reverses itself automatically, so no extra script is needed to undo the changes.
- The same approach can be reused on other tables and fields.

7.2 Limitations

- Client-side rules run in the browser, so they do not apply to records created through imports, web services or background scripts.
- The onCellEdit script blocks every list edit of State for all users, including those who may need it.
- Only the High impact case is handled; other impact levels have no special rules.
- The onChange script sets Urgency to High but does not reset it when Impact is lowered.

8. CONCLUSION

The Implement Client Script & UI Policy (Incident) project demonstrates how UI Policies and Client Scripts can work together to enforce dynamic field behaviour, automate updates and prevent incorrect data submission on Incident forms. By using `onChange`, `onSubmit` and `onCellEdit` scripts alongside UI Policy actions, the project ensures that fields like Assigned to and Urgency respond appropriately to changes in Impact, maintaining clean and consistent data. The solution is lightweight, efficient and easy to implement, and shows practical good practice for improving form usability and data integrity in ServiceNow.

Through this work, I gained hands-on experience with UI Policies, UI Policy Actions, the different client script types, the `g_form` API and systematic testing of form behaviour. These skills are useful for entry-level ServiceNow developer and administrator roles.

9. FUTURE SCOPE

- Add matching rules for other impact levels, such as making Description mandatory for Medium impact.
- Add server-side Business Rules or Data Policies so that the same rules apply to imports and integrations.
- Restrict the State list edit only for users without a specific role instead of everyone.
- Reset Urgency when Impact is lowered, using the else branch of the `onChange` script.
- Use UI Actions and `onLoad` scripts to guide users with additional messages on the form.
- Apply the same controls to the Service Portal and Workspace using UI Policy scripts and UI type settings.

REFERENCES

1. ServiceNow Documentation - UI Policies. <https://docs.servicenow.com>
2. ServiceNow Documentation - Client Scripts and the g_form API. <https://docs.servicenow.com>
3. ServiceNow Developer Site - Personal Developer Instance. <https://developer.servicenow.com>
4. Naan Mudhalvan (TNSkills) - ServiceNow course material. <https://naanmudhalvan.tn.gov.in>

APPENDIX A - CONFIGURATION SUMMARY

Table A.1: UI Policy and actions

Item	Value
UI Policy	High Impact Control (Incident, Active, Global)
Condition	Impact is 1 - High
Reverse if false / On load	True / True
Action: Assignment group	Mandatory = True; Read only = False
Action: Urgency	Read only = True; others left alone

Table A.2: Client scripts

Name	Type	Field	Key statement
Auto set urgency for high impact	onChange	Impact	g_form.setValue('urgency', '1')
Prevent save if Assigned To missing	onSubmit	-	g_form.showErrorBox(...); return false
Prevent state change via list edit	onCellEdit	State	callback(false)

APPENDIX B - RECORDS USED DURING TESTING

Table B.1: Records used for testing

Record	Details
New incident created	INC0010012 (Impact 1 - High, Assignment group Software, short description "testing UI policy")
Assigned to (successful save)	Beth Anglin
Reverse condition test	INC0010012 with Impact changed to 2 - Medium
List edit test	INC0008111 (State edited in the list)
Form-based update test	INC0008111 (State changed to In Progress on the form)